Project No: A3195-251



DEVELOPMENT OF AN ANDROID APPLICATION FOR FOOD DISCOVERY ON CAMPUS

Submitted by: Tan Jun Peng

Supervisor: Teh Kah Chan

School of Electrical & Electronic Engineering

A final year project report presented to Nanyang Technological University
in partial fulfilment of the requirements for the
Degree of Bachelor of Engineering

2026

Table of Contents

Table of Contents ii
Abstract iv
Acknowledgment v
List of Figures vi
Chapter 1 Introduction 1
1.1 Motivations 1
1.2 Objectives and Scope 2
1.3 Organizations 2
Chapter 2 Literature Review 3
2.1 Current Applications 3
2.2 Frontend Development Framework 6
2.3 Backend Development Architecture 7
Chapter 3 Ideation Process 8
3.1 Design Objectives 8
3.2 Frontend Ideation 10
3.3 Backend Ideation 13
Chapter 4 Frontend Development 15
4.1 Authentication Page 15
4.2 Home Page 17
4.3 EatWHAT 21
4.4 Stall Page 23
4.5 CookWHAT 27
4.6 Recipe Page 29
4.7 Profile Page 32
4.8 Admin Panel 36
4.8.1 Stall Management 36
4.8.2 Report Handling 38
4.9 Navigation Drawer 41
4.10 Animation 42
4.11 React Hooks 44
Chapter 5 Backend Development 45

5.1 Google Firebase 45
5.2 Backend Architecture 48
Chapter 6 Conclusion and Future Work 51
6.1 Conclusion 51
6.2 Recommendation in Future Work.....52
Reflection on Learning Outcome Attainment.....53
Acknowledging/Declaring the Use of GAI 54
References.....56

Abstract

Finding the full range of diverse food options within Nanyang Technological University (NTU) campus remains a challenge for students due to lack of current resources. This project aims to develop a collaborative mobile application, called NTUeats, that is designed for students to share food reviews and recipes with one another, enhancing food discovery and reducing uncertainty about meal choices, while strengthening the sense of community within NTU. The application will be developed with React Native and Expo for frontend development and Google Firebase for backend data management.

This report will mention the end-to-end development workflow, from aesthetic choices to high-fidelity Figma designs to code implementation. The application leverages on React Hooks to provide an interactive and functional interface, enabling users to review stalls and share recipes, as well as bookmark, comment and report within the interface itself to create a self-sustaining ecosystem with minimal effort from administrators. The application integrates Gemini AI to analyze reports and further reduce workload for administrators. Furthermore, the project will also focus on providing fun and engaging user experience through character-driven design, thoughtful animations and other aesthetic choices.

Acknowledgment

I would like to express my sincere thanks and great gratitude to my supervisor, Associate Professor Teh Kah Chan, for his continual support for the project. His regular monthly check-ins and valuable insights were instrumental in directing the execution of this project.

March 2026

List of Figures

Figure 1. Screenshot of NTU official website list of F&B options 3
Figure 2. Screenshot of Google Review 4
Figure 3. Screenshot of Cates application 5
Figure 4. The fonts chosen for type of content 8
Figure 5. Mascot Design 9
Figure 6. Iterative process of designing assets 9
Figure 7. The site-map structure for the application 10
Figure 8. The wireframe structure of the application 11
Figure 9. Figma mockup of the application 12
Figure 10. The database structure of the application 13
Figure 11. Login Page 15
Figure 12. Sign-up Page 16
Figure 13. Home Page 17
Figure 14. (a) The notification center with notifications. (b) The notification center without any notification. 18
Figure 15. (a) The EatWHAT snippet with swipe for more prompt. (b) The EatWHAT snippet that is swiped to the end to show "More Option" button 18
Figure 16. The CookWHAT snippet 19
Figure 17. (a) Randomizer first phase with “Eat”, “Cook” and “Both Also Can” options. (b) Randomizer second phase with "thinking" mascot. (c) Randomizer third phase with decided stall or recipe. 20
Figure 18. EatWHAT page 21
Figure 19. Filter for EatWHAT page 22
Figure 20. The Stall Page layout 23
Figure 21. Image gallery modal of images from reviews 24
Figure 22. Menu modal 24
Figure 23. Section for users to insert a review 25
Figure 24. Base of the feed with “View More Reviews” button. 26
Figure 25. CookWHAT page 27
Figure 26. Filter for CookWHAT page 28

Figure 27. Recipe Page at default state 29
Figure 28. Steps section of Recipe Page 30
Figure 29. Comments section 31
Figure 30. Upload comment overlay for Recipe Page 31
Figure 31. Profile Page default layout 32
Figure 32. Profile Page with Favorite section selected 33
Figure 33. (a) Profile Page with Recipe section selected (b) Modal for editing comments 34
Figure 34. Upload Recipe Page 35
Figure 35. Profile Page with Comments section selected 35
Figure 36. Admin Dashboard layout 36
Figure 37. Admin Dashboard for Add New Stall 37
Figure 38. Admin Dashboard for Stall List 37
Figure 39. Report Page 38
Figure 40. Admin Dashboard for Report List 39
Figure 41. Filter options for Report List 40
Figure 42. Navigation Drawer 41
Figure 43. Mascot Loader sprite sheet 42
Figure 44. Code for General Page Animation showing the animation applied 43
Figure 45. Code for Home Animation showing the animation applied 43
Figure 46. Cloud Firestore database 45
Figure 47. Image storage for EatWHAT 46
Figure 48. Example of report document, where text refers to the report text input by user and the analysis refers to the interpreted output generated from the AI model. 47
Figure 49. Backend services utilities 48
Figure 50. Code snippet of StallServices.js that shows the algorithms within the getter function 49
Figure 51. Code snippet for LikeService.js for fetching total likes for an entity 50

Chapter 1 Introduction

1.1 Motivations

Nanyang Technological University (NTU) offers a diverse range of dining options on campus with 13 canteens, each hosting multiple food stalls. However, incoming students often face difficulty exploring the full range of available options. Moreover, students tend to eat at the same canteen repeatedly as most would consider convenience as one of the key determinants of where they would dine [1], limiting their awareness of stalls in other canteens. In addition, research has shown that people are inclined to have a greater number of choices, as it provides them freedom in decision-making [2]. Similarly, studies have shown that most students would prefer canteens that provide a wider variety of food options [1]. A platform that has a collective list of all the food stalls in NTU would align with these preferences and address such issues by allowing for visible access to the full range of dining options.

Many youths discover new dining options through social media or online searches [3], but existing resources provide only partial information. For instance, the official NTU website lists the location of stalls but lacks detailed descriptions, Google Reviews offers reviews on canteens only without focusing on individual stalls, and platforms such as Cates focus on food ordering, displaying only affiliated vendors without coverage of reviews. Social media platforms, such as Instagram and TikTok, allow for influencers to provide recommendations as well, but the young tend to be cautious of such suggestions as they may be biased or misleading due to sponsorship and affiliation [3], and such platforms do not provide extensive coverage. This results in a superficial understanding of available foods within campus from online resources.

To address these limitations, the project aims to leverage a community-driven platform, where students can share food recommendations and review specific stalls. This creates an unaffiliated platform, where students can get trustworthy recommendations of individual stalls free from commercial bias.

Additionally, students can also upload recipes for students who cook. Those on campus often have limited cooking equipment and generally want to budget their ingredients carefully [4]. A recipe sharing application tailored for students would address these needs, as other students would likely be facing similar circumstances and can share recipes that are more practical for the community.

1.2 Objectives and Scope

The objective is to create a collaborative mobile application where students can share food reviews and recipes with one another, thus enhancing food discovery and reducing uncertainty about meal choices, while strengthening the sense of community within NTU. Data will be collected within NTU, content generated will be simulated, and the targeted users will be NTU students. This will not extend to external eateries, and commercial partnerships.

1.3 Organizations

The report will be organized into 5 chapters:

- **Chapter 2:** In this chapter, we will investigate the current applications that are available, objectives for the application and the reasons for the tools used for frontend and backend development.
- **Chapter 3:** In this chapter, we will describe the ideation process of developing the application, which shows the planning phase for aesthetic choices, frontend interface and backend architecture.
- **Chapter 4:** In this chapter, we will showcase the application design and frontend development of the application.
- **Chapter 5:** In this chapter, we will showcase the backend development of the application.
- **Chapter 6:** In this chapter, we will conclude the report and propose possible future additions to the application.

Chapter 2 Literature Review

2.1 Current Applications

The current ways students can check for stall information within NTU campus would be through NTU official website, Google Reviews and the Cates Application.

The NTU official website provides a list of canteens and some food stalls within campus. It shows the opening hours and how to get there, through NTU maps functionality. This can be seen in Figure 1.

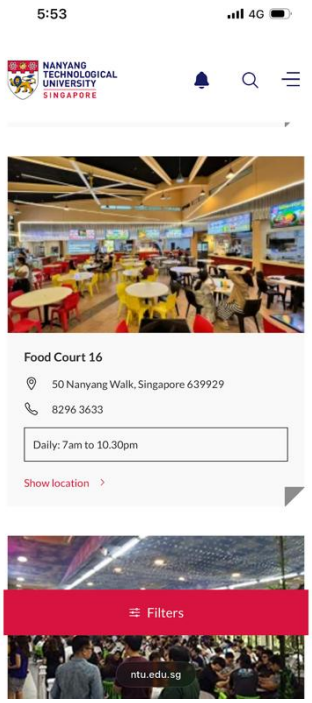


Figure 1. Screenshot of NTU official website list of F&B options [5]

This can benefit students by showing them how to get to the different canteens. However, the website lacks details of what food is sold and the general preferences for it, leading to users having little information about food quality and availability.

Google Review provides a platform for users to review and rate the canteens. The list can be found in Figure 2.

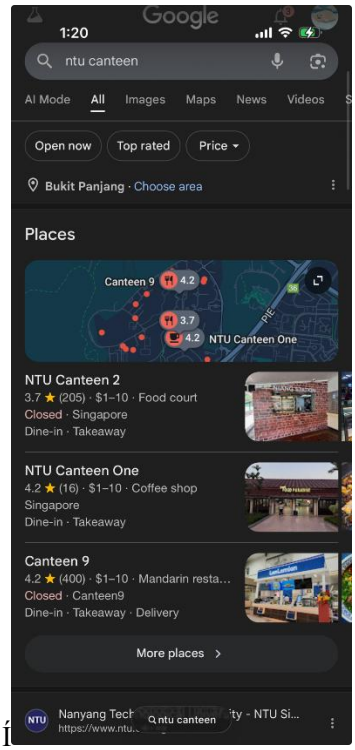


Figure 2. Screenshot of Google Review [6]

This platform provides an adequate list of food recommendations from the stalls, but the reviews are all stored under the canteen and not the specific stalls. Users will have to sift through multiple reviews of varying stalls to get the information for the relevant food they want.

The Cates application is a food ordering platform within NTU that shows the various stalls and menu options.

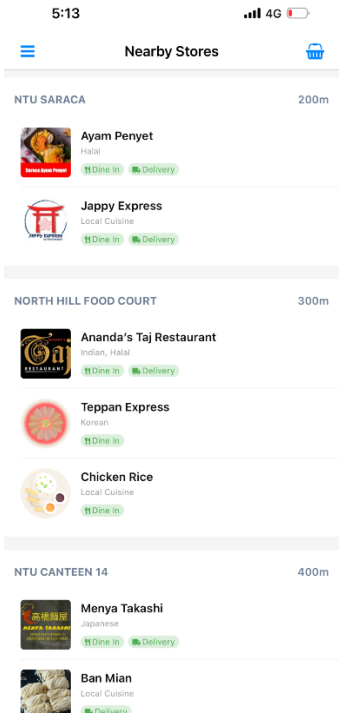


Figure 3. Screenshot of Cates application [7]

The application provides a good method to understand the various food choices from the stalls within the platform, with an additional option to order. However, the application only shows affiliated stalls that utilize their food ordering services. This narrows the food choices, as can be seen in Figure 3 where the application only shows 2 stalls within Saraca canteen, but there is a total of 8 stalls.

This shows the limitations of current applications as there has not been a complete list of food options within NTU. NTU official website only shows the canteens available, Google Reviews are limited to showing canteens reviews – and not for specific stalls – and the Cates Application only shows a restricted list of affiliated vendors.

2.2 Frontend Development Framework

The frontend development uses React Native with Expo. React Native has numerous benefits for mobile development, as it provides a framework with useful inbuilt and external features [8].

Firstly, React Native uses a single JavaScript codebase to render components, allowing for development for both IOS and Android applications simultaneously. The use of JavaScript, and the fact that it is open sourced, also allows for development to tap into a large developer community, benefiting from pre-existing libraries to aid development. For example, there is “react-native-firebase” which allows for industry standard integration of backend development from Firebase to frontend components.

The use of Expo framework allows for better workflow when developing the application. It allows a developer to focus on JavaScript code and settles the native configurations for IOS and Android. It also creates a QR code for developers to test the application on their phone, reducing the need for a complete build before testing it on the phone interface.

React Native also allows for rapid iteration of components and design. This includes Hot Reloading, which allows developers to see code changes in real-time, without the need to restart the application on each update. The project also incorporates NativeWind, which allows developers to adjust styling of elements through intuitive classes, further helping with rapid iteration of styling, without the need for an external style sheet.

Lastly, React Native allows for implementation with React Hooks, which facilitates efficient state management and lifecycle handling. The main hooks that I utilize would be useState, which allows for management of local component states that can be shared and updated by other components, and useEffect, which handles asynchronous data and function handling, allowing components to be dynamic.

2.3 Backend Development Architecture

The backend development uses Google Firebase for data and image storage through Cloud Firestore and Cloud Storage. Firebase was chosen for its beneficial all-in-one system that provides numerous useful features for storage [9].

Firstly, Firebase is a platform that uses a Backend-as-a-Service (BaaS) model, which hosts the backend directly from their server. This ensures the project can benefit from security, scaling and maintenance of server provided by Google, allowing development focus to remain on frontend user experience and other features.

Cloud Firestore handles storage of data. Firestore offers real-time synchronization of data, ensuring users to get updated information immediately in the application. Firestore is also a NoSQL database which allows for addition of new fields without needing to undergo complex data migrations.

Cloud Storage allows images to be easily uploaded and stored within the cloud securely. The images stored have access tokens, which are a unique URL to access the images. These tokens can be stored within the database to allow for a seamless link between the structured data and the image.

Moreover, Firebase has Machine Learning capabilities that can be used to enhance other features.

Chapter 3 Ideation Process

3.1 Design Objectives

Given that my target audience are students in NTU, the application was designed to have a more fun and playful aesthetic to make the application more engaging and informal so students are more encouraged to explore the platform's features.

The name of the application will be NTUeats, which is derived from the phrase "where to EAT in NTU". To establish a strong cultural connection with the Singapore-based student body in NTU, the application will be adopting a Singlish-inspired nomenclature, such as the naming of the stall-sharing component to "EatWHAT" and the recipe-sharing component to "CookWHAT". This approach will make the application feel more relatable, increasing user engagement within the students [10]. Moreover, the "WHAT" serves as a consistent brand identity for the various features in the application.

The color selection was a key factor in the aesthetic choice. The primary colors chosen were red and green, as red is known to make people feel hungry, while green is known to resemble healthy food choices [11]. Additionally, the use of off-white cream as the background color created a softer and more relaxed tone [12].

The typography was also another key factor. 'Koulen' font was chosen for the header as it looks strong and bold. 'Inter' was chosen for the body text as it was clear and legible even at small sizes. 'Ranchers' was chosen as a playful accent font as it was rounded and energetic, which can be used to reinforce the friendly and engaging identity of the application. These fonts can be seen in the figure below.



Figure 4. The fonts chosen for type of content

This playful aesthetic is further implemented through integration of cartoon-style characters. I chose

to design and create cat-theme characters as a mascot for the application as shown in Figure 5. This mascot served to both make the application more memorable, as well as humanize the brand, by incorporating engaging and human-like actions to the mascot [13].

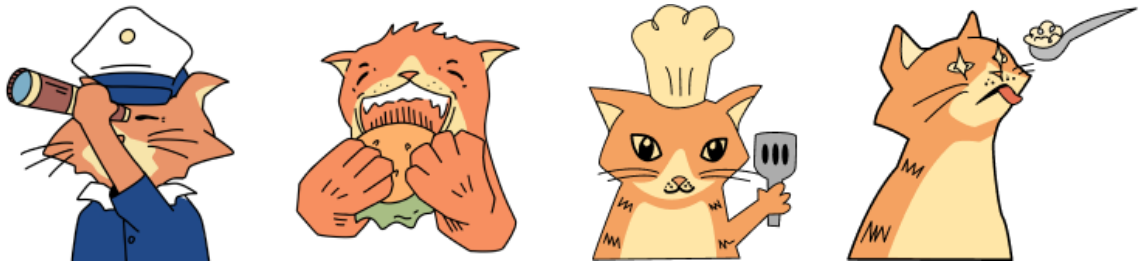


Figure 5. Mascot Design

The characters were designed on Affinity Designer, a digital illustration software, and their purpose was to bring life to the application. The specific theme was selected due to the popularity of cats among students in NTU, as evident from the large following of the “meowsofntu” Telegram group, which posts sightings of cats within the campus, with more than 10,000 followers.

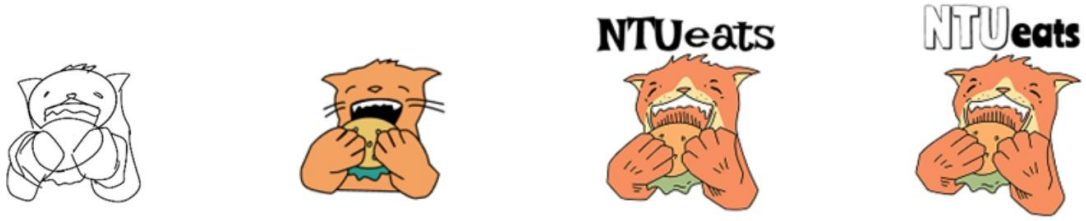


Figure 6. Iterative process of designing assets

Moreover, iterative creation of assets, consisting of sketching, testing of color schemes and typography, was carried out to ensure readability and compatibility of design, as shown in Figure 6.

3.2 Frontend Ideation

The creation of frontend interface started with creation of a sitemap, followed by wireframes of the application layout, then a Low-fi mock-up of the application. These designs were created in Figma.

The sitemap structure was structured to outline the required pages and the links between each page. This is shown in Figure 7.

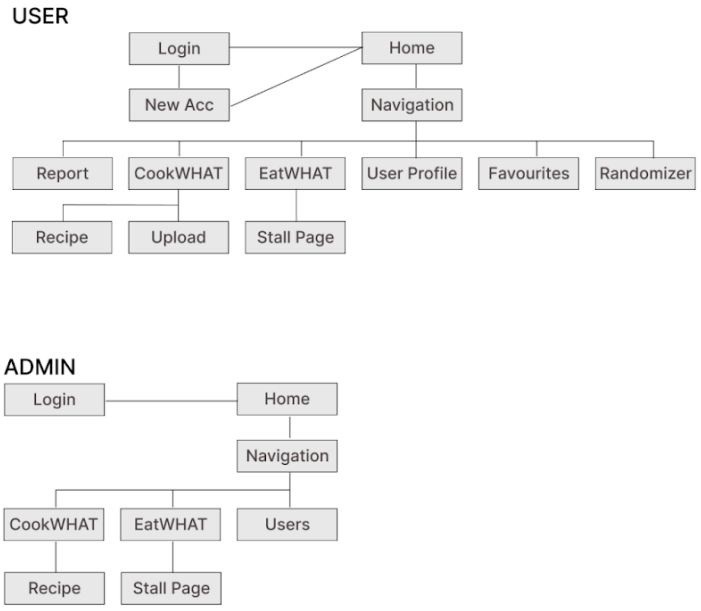


Figure 7. The site-map structure for the application

The main objective of the sitemap is to ensure the user does not require many clicks to get to his desired page. The general aim for the application relied on the 3-Click Rule, where users should be able to find what they need in 3 clicks. This is shown in Figure 7, where users can get to most of the pages withing 3 components from the Home page.

The sitemap lists the general pages that were aimed at being developed and indicates the distinction between those pages the user has access to and the pages the admin needed to have.

Wireframes were created to establish the general layout and visual structure of the application. A storyboard was created to show the links between pages through navigation inputs. The blue arrows indicate the button and page the button takes the user to.

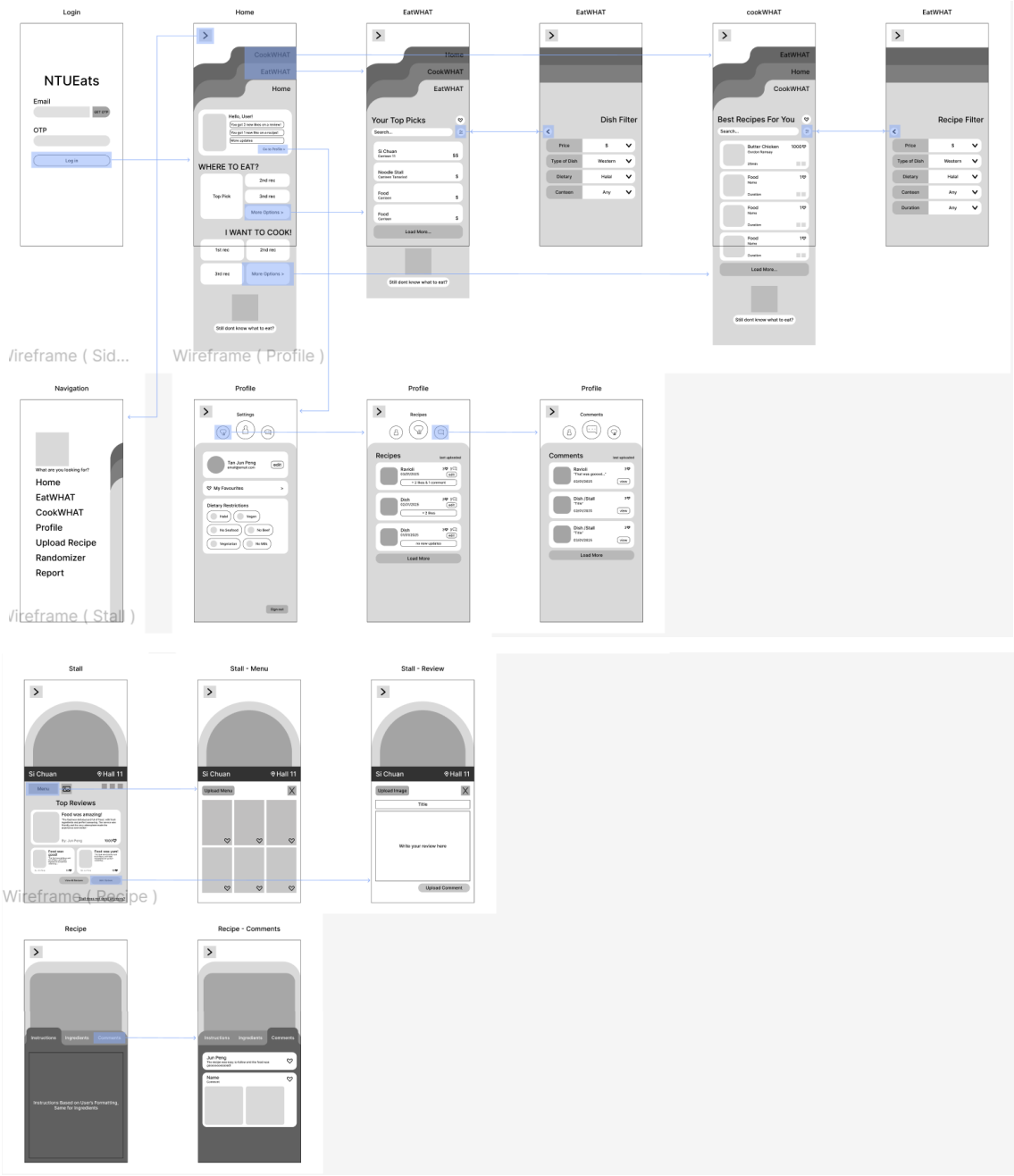


Figure 8. The wireframe structure of the application

Following the planning and creation of the aesthetic for the application, a low fidelity mock-up of the

application was created, as shown in Figure 9. This stage was crucial to test the arrangement of the components, such as font, artwork and colors, in the application. Images had been initially omitted, as they will differ between people, hence the focus remained on the structural layout of the application. This allowed for rapid iteration on component placement before proceeding to high-fidelity development.



Figure 9. Figma mockup of the application

3.3 Backend Ideation

The backend ideation was developed with Figma. This is the planning phase to design the data required by the application, and the relationships between them.

The visualization of the database is shown in Figure 10. The database had to ensure logical relationships between all tables, and ensured the data are relevant to the application's core functionalities.

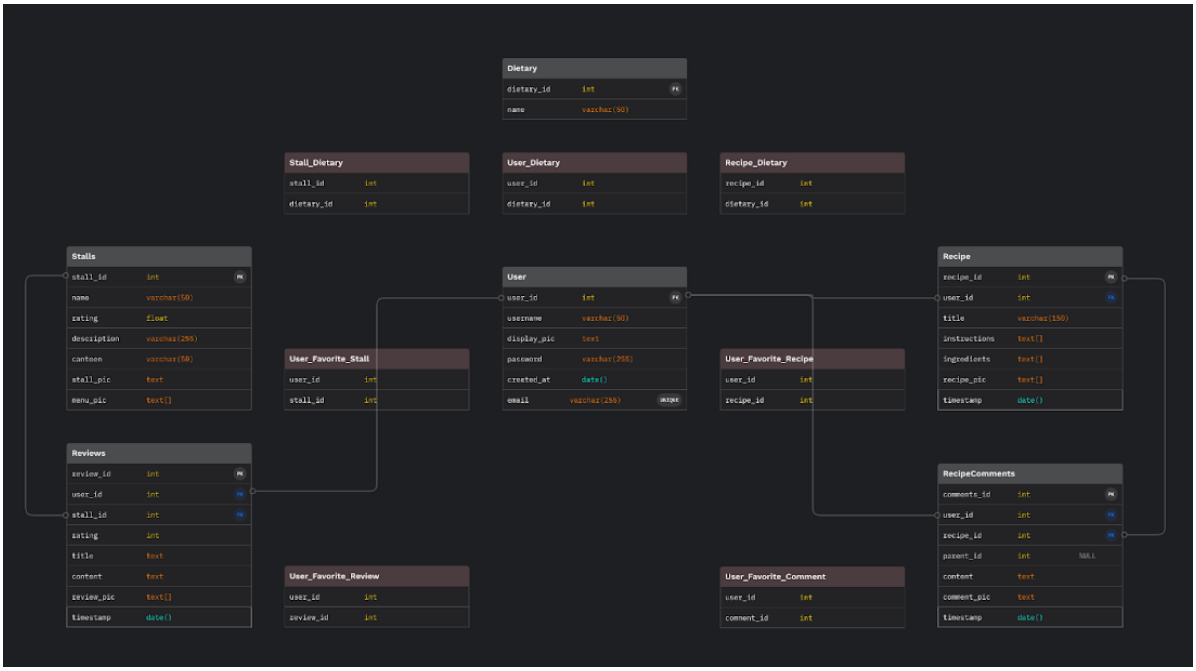


Figure 10. The database structure of the application

The database will have 5 main tables (User, Recipe, Recipe Comments, Stalls and Reviews). It follows the structure:

- All data in the tables will have its own unique primary key.
- Each Recipe will have the User ID as a foreign key.
- Each Recipe Comment will have User ID and Recipe ID as a foreign key.
- Each Review will have the User ID and Stall ID as a foreign key.
- There will be a junction table that links dietary preference to User, Recipe and Stall that shows the dietary restrictions of users, recipes and stalls respectively.
- There will be a junction table that links User to Recipe, Recipe Comments, Stall and Reviews that shows the recipes, comments, stalls and reviews that the user has liked.

This will mean that User and Stall tables are the only tables that do not rely on other tables. This must be handled with User sign up and Admin adding the stalls. The other tables reference the User ID to ensure the data is traced back to the individual user.

Chapter 4 Frontend Development

4.1 Authentication Page

The application starts the user at the login page as shown in the figure below. The interface features the illustration of the NTUeats mascot consuming a burger, humanizing the mascot and creating a visually engaging design. This establishes the character-driven brand identity from the first touchpoint.

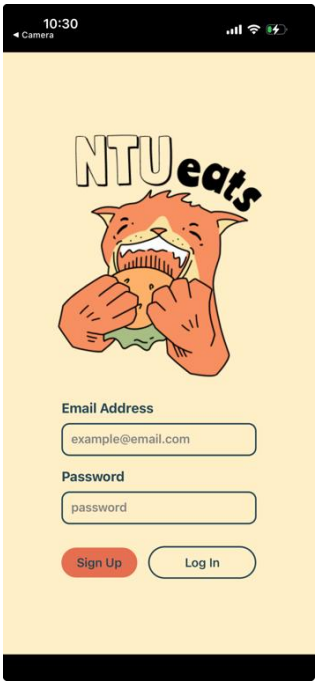


Figure 11. Login Page

The login screen creates a straightforward workflow for users to key in their information. A “Sign Up” button is styled in high-contrast red to create a cleat call-to-action (CTA) for new users, where clicking it will take users to the sign-up page. Red also stands out as a visual anomaly compared to the other primary input fields, frequent users will learn to ignore it [14], allowing them to focus instinctively on the normal login workflow.

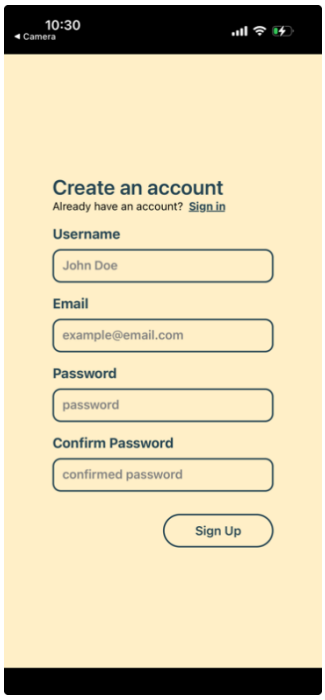


Figure 12. Sign-up Page

The sign-up page allows users to create an account. The form has strict validation protocols to ensure data integrity and account security. The application requires user to input a proper "name@domain.com" format for emails and adopts a strong password policy by ensuring the password chosen is complex. Upon clicking on the "Sign Up" button, the system will send a verification email to user to ensure authenticity of the user's email address. The account is only activated once user keys in the verification numbers sent to their emails.

The backend for the authentication uses Clerk, which is an authentication and user-management platform. Clerk handles the creation of accounts, security of password and session management for users, ensuring user login and sign-ups are handled securely.

4.2 Home Page

Upon successful authentication, users are directed to the Home Page, which consists of a navigation at the top of the page, a dynamic notification center, highlights section, and a randomizer.

The main feature of the application will be to showcase the EatWHAT and CookWHAT pages. This is emphasized in the Home Page, where the top navigation provides easy access to these components, as can be seen from the top right of Figure 13. The design of the navigation resembles physical folders, where users can pretend to "flip through" the sections, creating a fun yet organized approach to discover content.

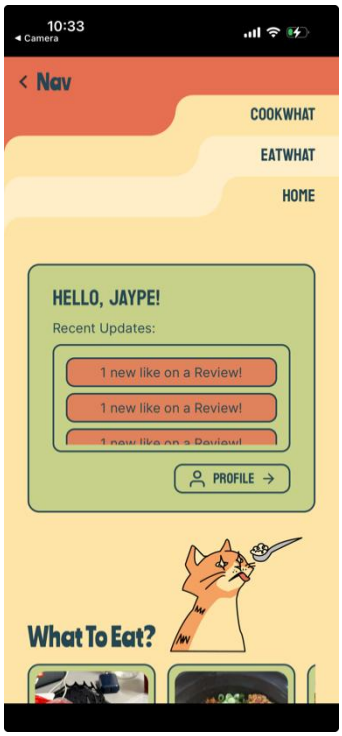


Figure 13. Home Page

The header of the Home Page would be the notification center shown in Figure 14. It aims to create an immediate sense of community by providing personal salutation, greeting the user by name upon login, and displays a real-time update of interactions, such as new likes on user's review, recipe or comment. Each notification is interactive, where clicking on it redirects user directly to the relevant review or recipe. It also specifies the number of new likes and exact content type affected, ensuring user remains informed of latest activity immediately when he enters the application. Additionally, the Profile button is conveniently located in the header to allow users to easily access their personal settings.

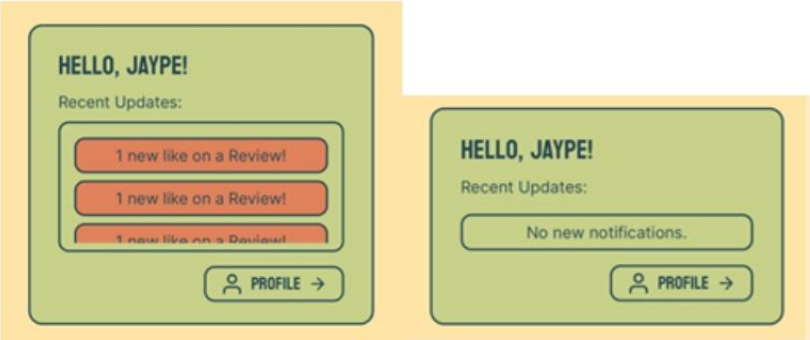


Figure 14. (a) The notification center with notifications. (b) The notification center without any notification.

The highlights section serves as a preview of EatWHAT and CookWHAT section, designed to allow users to gain a brief overview of these sections, encouraging them to further explore through intuitive interactions.

The segment showcases a snippet of the EatWHAT section, displaying the most favorited stalls under the heading “What To Eat?”. The list displays the first 2 stalls fully following a partially visible portion of the third, indicating the list is scrollable. A guiding text encourages users to swipe for more options. The text is programmed to disappear after the user swipes, reducing visual clutter. To prevent the Home Page from being over saturated, the list only shows the top 4 highlights, prompting users to go to the EatWHAT section through the “More Options” button at the end of the list as shown in the figure below.



Figure 15. (a) The EatWHAT snippet with swipe for more prompt. (b) The EatWHAT snippet that is swiped to the end to show "More Option" button

The highlights continue with the snippet of the CookWHAT section, under the heading “I Want To Cook” This section has similar design framework to EatWHAT snippet above, allowing users to understand the sections easier.

To further reinforce the playful aesthetic, the header of both segments features a custom illustration of the NTUeats mascot. In the EatWHAT snippet, the mascot’s eyes are lit up while its being fed, signifying that it is hungry and ready to eat. In the CookWHAT snippet, the mascot is wearing a chef’s hat while holding a spatula, signifying that the cat is ready to cook its meal. This approach further establishes character-driven branding by providing a friendly, relatable companion for the user.



Figure 16. The CookWHAT snippet

The final section of the page features a Randomizer, designed to solve the food choice for users that still have difficulty deciding. The section provides them with three options: “Eat”, “Cook” or “Both Also Can” – a Singlish-inspired option for those who are open to either option. Upon selecting an option, the segment transitions to an engaging animation, with 2 distinct states. Firstly, the mascot appears with an exaggerated “thinking” expression while the card spins rapidly, simulating the data retrieval process in a playful manner. The end sequence for the animation is the mascot presenting the chosen stall or recipe like a magic trick reveal. The mascot expression changes to a more confident and bolder look, providing a sense of closure and satisfaction that he has chosen a good choice. The user is presented with a choice to select the result or retrigger the randomizer.

The randomizer uses a Conversational User Interface (CUI) approach, where the text is written to mimic an informal conversation. The mascot is also given exaggerated expressions to further humanize it. The choice of words and mascot expression makes the interaction feel more like a conversation with a friend.



Figure 17. (a) Randomizer first phase with “Eat”, “Cook” and “Both Also Can” options. (b) Randomizer second phase with "thinking" mascot. (c) Randomizer third phase with decided stall or recipe.

To ensure a more intuitive user experience, a consistent green color is applied to the interactive elements, within the Home Page, to allow the user to instinctively identify them.

4.3 EatWHAT

The EatWHAT page serves as a comprehensive directory for food stalls within NTU, where its aim is to help students discover the food on campus. The page header features the same mascot as the one in the Home Page, alongside the greeting “What are we eating today?”. This makes searching for food feel guided. Each stall is represented by a Stall Card, which shows a summary of the stall. The Stall Card will show an image of the stall, and provide core information, such as the stall name, general dishes, canteen location and relative price indicator. Moreover, clear icons for Halal and Vegetarian options are displayed on the Stall Card, allowing users to verify the stalls’ dietary restrictions.

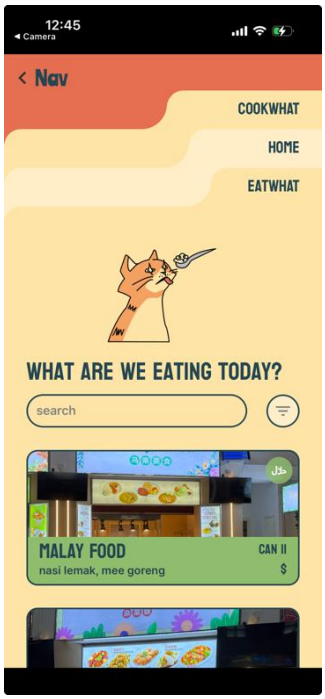


Figure 18. EatWHAT page

By default, the stalls are arranged in a random order to allow for food discovery. For users with a specific stall in mind, a prominent search bar allows users to find them. When the search bar is active, the page is programmed to scroll to shift the search bar to the top of the page, which allows users to see what they are typing, and have a glimpse of the filtered search result while their keyboard is active. The order of the randomized arrangement is also stored in the local session state, so that users would still see the original “randomized” arrangement after clearing their search query. This would allow them to easily relocate stalls they had previously seen before initiating the search

Users can further filter their result through a filter dropdown located at the side of the search bar, as

shown in Figure 19. The filter allows users to sort the stalls by popularity, price and canteen. Sorting by price allows for the options shown to fit within the student's budget, while sorting by popularity highlights the popular choices. Sorting by canteen allows users to see options that are convenient for them to go to.

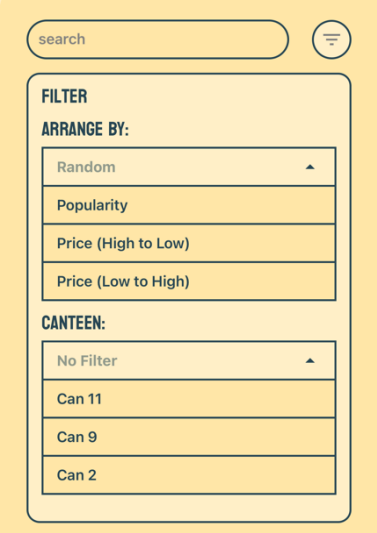


Figure 19. Filter for EatWHAT page

4.4 Stall Page

Upon selecting a specific stall from the EatWHAT list, the user is brought to the Stall Page, as seen in Figure 20. This page consists of three primary features, which are the Header, the Interactive Row and the Community Review Feed.



Figure 20. The Stall Page layout

The Header shows a brief overview of the stall selected. This includes the name of stall, stall location, and an image of the stall. To verify dietary restrictions, such as Halal or Vegetarian options, clear icons are shown in an overlay at the bottom-right of the stall image. An intuitive “X” icon is positioned at the top corner of the screen, to allow for back tracking to the previous page.

The Interactive Row contains four primary buttons, which includes the Favorite, Image Gallery, Menu and “Add Review”.

The Favorite button allows users to bookmark stalls, saving them in their profile for future sessions. This interaction will be used to aggregate the stalls popularity as mentioned in the “popularity” filter in EatWHAT.

The Image Gallery button triggers a modal overlay that shows all images of food reviews as shown in Figure 21, which is sorted by the most “Likes” on the reviews. This allows users to get a quick view of food choices without needing to scroll through wordy reviews.

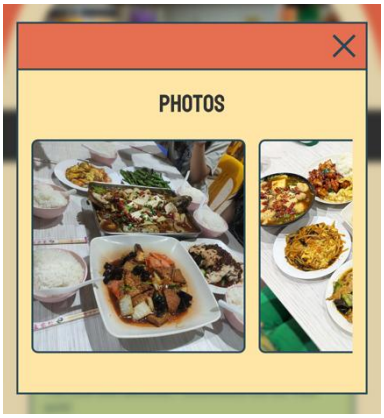


Figure 21. Image gallery modal of images from reviews

The Menu button triggers a modal overlay as shown in Figure 22, displaying a scrollable list of stall menus taken by other users, where each menu can be liked by other users. The menu images are sorted by “Likes”, ensuring the most popular menus are sorted first. An “Upload Menu” button at the base of the modal allows user to upload pictures of menus. As the menus of stalls changes occasionally, due to price changes, changes in food or rebranding, it is difficult for an admin to keep up with every change. This functionality allows for data integrity of stall menu, as the community will provide the real-time updates to menu, and “Like” the menus that are relevant, creating a system that is self-regulating.

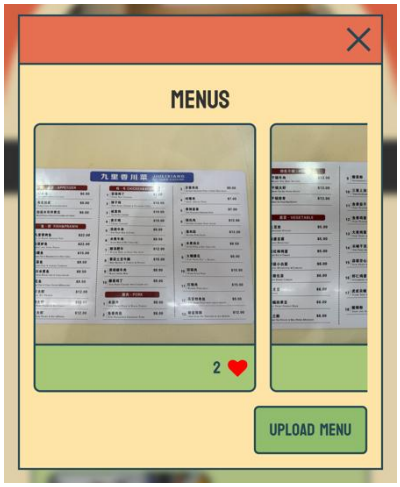


Figure 22. Menu modal

To ensure the menu and gallery images are legible, a custom Simple Viewer component was developed. This viewer supports pinch-to-zoom gestures and allows users to move around the image to inspect the fine details. This allows for small texts on physical menus to be seen and allows users to greater analyze the food options, that would normally be lost on the small standard mobile screens.

The “Add Review” button transitions the page to a form as shown in Figure 23. The form allows users to submit a review of the stall, by providing title of their review, description of the stall or meal, and include a photo. This would provide the data for the stall, so other users can understand the quality of the food from the stall.

The “Add Review” button changes its label to “Cancel” and changes from green to red, and vice versa, when clicked. This creates a signifier for the user to be aware of the current state of the application and allows user to easily navigate back to stall details.

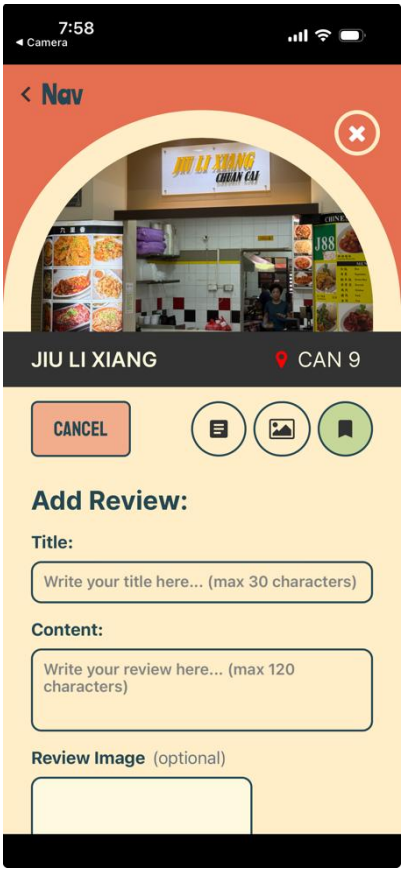


Figure 23. Section for users to insert a review

The forms in the application are standardized with reusable components, such as the input field and an image uploader. This is to ensure visual consistency throughout the application and minimize redundant code duplication. The image uploader component was developed to create smooth and consistent image uploading workflow for users, it leverages the Expo Image Picker to select media from the device gallery and extracts the image URI to be uploaded to Firebase Storage.

The upload button also disables itself after initial clicking, preventing duplicate entries. It also transitions its text to a “Loading” state, before providing real time feedback through redirecting users to their previous page and signify successful upload through an alert popup.

The next section of the page is the Community Review Feed which shows user feedback about their meals from the stall, as shown in Figure 20. Each review displays the post date, author and other detailed content as mentioned in the “Add Review” segment. Users can “Like” reviews by tapping the heart icon at the bottom-right of the review. The reviews can be sorted by “Most Liked” or “Most Recent”, letting users decide if they want to get the popular reviews or updated ones.

To optimize performance of the page, the page undergoes “Lazy Loading” by loading only 4 most relevant reviews. A custom “View More Reviews” button is located at the base of the feed to allow for additional content to be retrieved as shown in Figure 24.

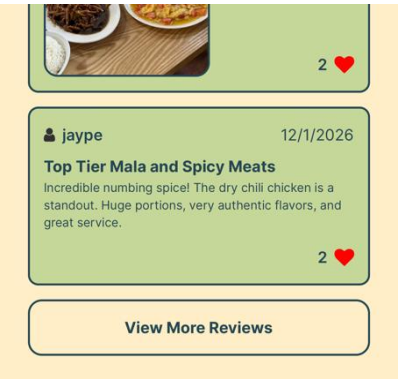


Figure 24. Base of the feed with “View More Reviews” button.

4.5 CookWHAT

The CookWHAT page serves as an alternative food option for students, where it targets students who prefer to cook. To ensure the platform remains intuitive, the layout of the page is almost identical to the EatWHAT page. The page header features the same mascot as the one in the Home Page, alongside the greeting “What are we cooking today?” Each recipe is represented by a Recipe Card, which includes the recipe name, publisher of the recipe, cooking duration and an image of the dish. The icons for Halal and Vegetarian options are also displayed on the Recipe Card, like the Stall Card.

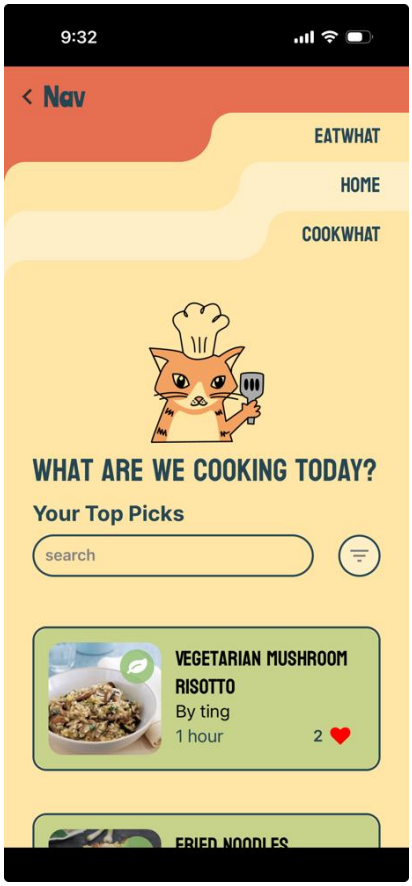


Figure 25. CookWHAT page

By default, the recipes are arranged in a random order as well, with a prominent search bar at the top. The search bar has similar “scroll to top of the page” functionality when active and the randomization state is also stored in the local session.

The filter dropdown for CookWHAT can sort the list by popularity and recency, allowing users to choose to try community favorites or test newer entries.

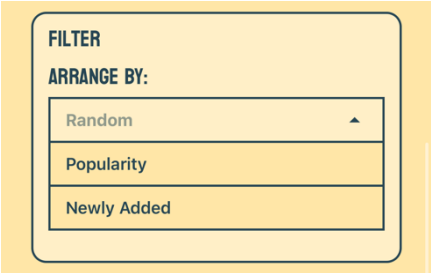


Figure 26. Filter for CookWHAT page

4.6 Recipe Page

Upon selecting a specific recipe from the CookWHAT list, the user is brought to the Recipe Page, as seen in Figure 27. The page consists of three distinct parts, which are the Header, the Sub-Navigation Bar and the dynamic Content Area.

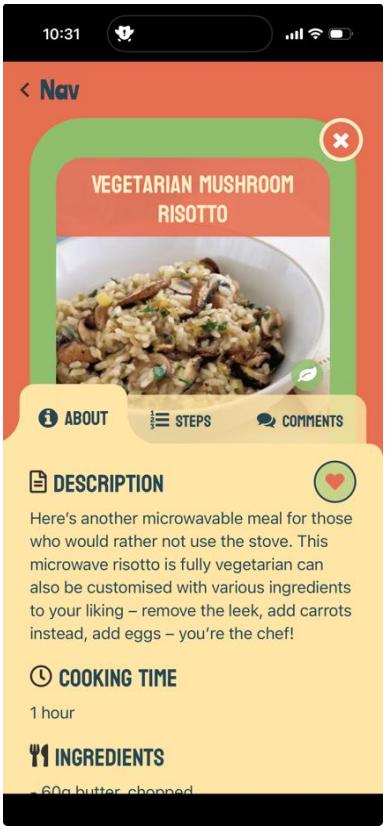


Figure 27. Recipe Page at default state

The Header shows a brief overview of the recipe selected. This includes the name of dish and an image selected by publisher. To verify dietary restrictions, clear icons are shown in an overlay at the bottom-right of the image. The same "X" icon is positioned at the top corner of the screen, like the Stall Page.

The Sub-Navigation Bar consists of 3 different options, which are labelled "About", "Steps" and "Comments". This bar allows users to toggle between three different views, where the Content Area changes according to the option selected.

The About section is the default state for the Content Area, as shown in Figure 27, and it provides

information of details about the dish. This includes description of the dish, cooking time to prepare and ingredients needed for the dish. This helps the user to evaluate and plan for the dish, both in terms of time commitment and ingredients needed. There is also a “Like” icon at the top-right of the section to allow user to bookmark the recipe for future references.

The Steps section provides the instructions for the dish as shown in Figure 28. It aims to provide maximum clarity for the user when cooking the dish. The header clearly states the total number of steps needed to cook the dish, serving as a preempt for users to gauge the complexity of dish before starting. The instructions are displayed in a step-by-step format, ensuring it is easy to follow.



Figure 28. Steps section of Recipe Page

The Comments section allows users to provide feedback on the dish as shown in Figure 29. Users can share their personal experiences or inputs for the dish. To ensure the most helpful feedback is emphasized, comments are organized based on most “Likes”.

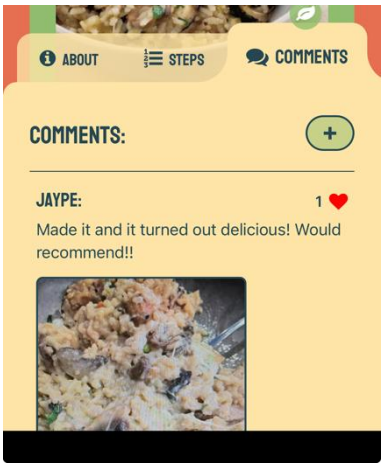


Figure 29. Comments section

The user can add a new comment with the prominent plus icon at the top right of the section, which will trigger a modal overlay, as shown in Figure 30, that users can input textual content and upload an optional image of their prepared meal.



Figure 30. Upload comment overlay for Recipe Page

4.7 Profile Page

The Profile Page is a personalized dashboard for each user to allow them to manage their account and track past interactions with the application as shown in Figure 31. The interface is divided into two main components, the sub-navigation bar in the header and the dynamic content area. Consistent with the application other pages, the recognizable “X” icon at the top right to allow for backward navigation.

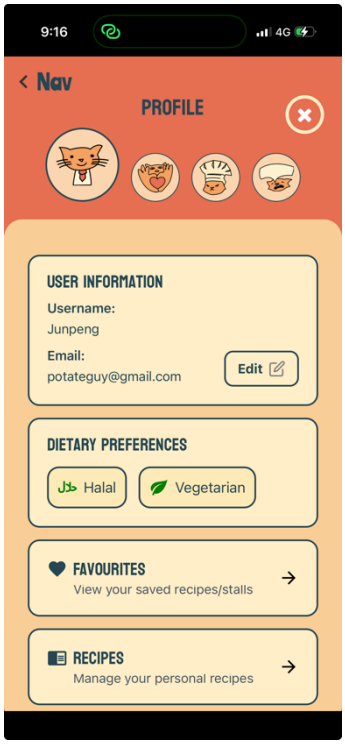


Figure 31. Profile Page default layout

The sub-navigation bar allows users to toggle between four different sections, General Profile, Favorites, Recipes and Comments. To maintain the application character-driven branding, each navigation tab contains simple illustrations of the NTUeats mascot. For example, the “Favorites” tab depicts the mascot creating a heart shape with his hands, and the “Recipe” tab depicting the mascot in a chef’s hat. These icons were designed to be minimalist, so that they fit neatly in the header while remaining intuitive for users.

The General Profile section handles account settings, allowing users to update their email, username and password. The user can also update their dietary preference, current supporting only Halal and Vegetarian options. Selecting a dietary restriction will toggle a global application filter that makes the application exclusively display stalls and recipes that comply with selected restrictions, ensuring a safe

and tailored browsing experience. To assist new users that may not understand the icon-based sub-navigation, navigation buttons with clear labels and descriptions are added at the bottom of this section.

The Favorites section shows a list of stalls and recipes bookmarked by the user as shown in Figure 32. The user can switch between “Favorite Stalls” and “Favorite Recipes” through the green toggle button. The list contains cards that provide routing to the respective Recipe or Stall Page.



Figure 32. Profile Page with Favorite section selected

The Recipes section shows the list of recipes created by users as shown in Figure 33. Each recipe card displays core metadata of the recipe, such as name, description, date created and the image of the recipe. The user can edit and be directed to selected Recipe Page through the buttons on the card. The edit function will trigger a modal with fields that are pre-populated with recipe’s existing data, as well as a delete button, facilitating updates or removal of recipes.

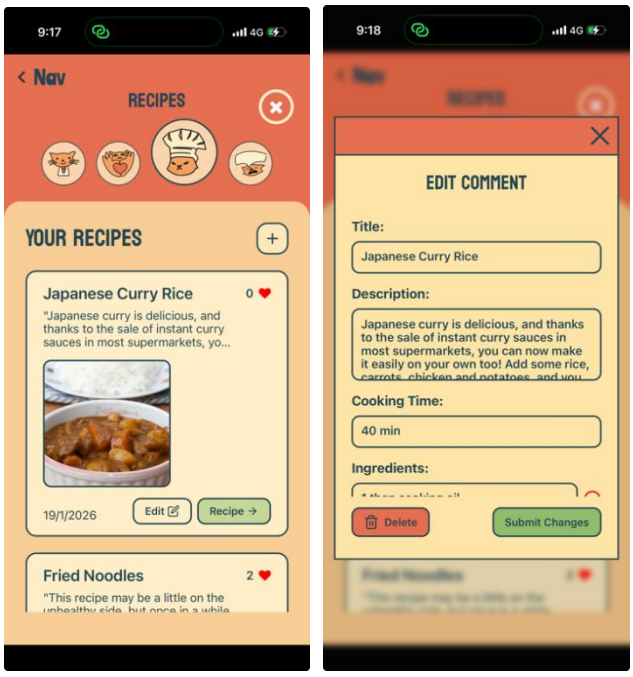


Figure 33. (a) Profile Page with Recipe section selected (b) Modal for editing comments

The page has a “plus” button at the top right that routes user to the Upload Recipe Page as seen in Figure 34, a comprehensive form for user in input the title, description, cooking time, dietary restrictions and a cover image. Since number of ingredients and instructions differ between recipes, a custom dynamic input list component was developed. This component manages flexible arrays of inputs, where users can add and remove input fields, utilizing color-coded icons, blue “plus” icon for addition and red “minus” icon for deletion.

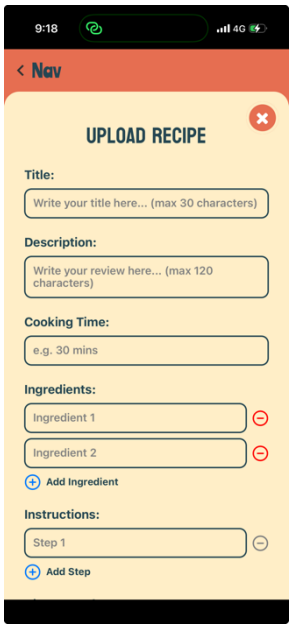


Figure 34. Upload Recipe Page

The Comments section shows the list of feedback generated by user, encompassing recipe comments and stalls reviews, as shown in Figure 35. A toggle button switches between two types of comments, like the Favorite section. The comment card shows metadata of the comment, like the recipe card. Users can also edit past submissions and navigate to the recipe or stall that the comment is on through the buttons at the bottom of the comment card.



Figure 35. Profile Page with Comments section selected

4.8 Admin Panel

The Admin Panel serves as a centralized dashboard that allows administrators to maintain the application and oversee content updates. This is strictly for admin access only, where access is governed by Role-Based Access Control (RBAC). Access is blocked through an access token in the backend, preventing unauthorized manipulation of databases.

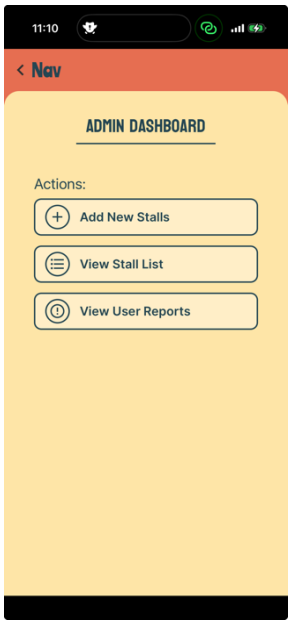


Figure 36. Admin Dashboard layout

4.8.1 Stall Management

The primary function of the current admin dashboard is stall management, where admin is given access to the full CRUD (Create, Read, Update, Delete) capabilities. This is implanted through the “Add New Stall” and “Stall List” interface.

The “Add New Stall” interface is a comprehensive form designed for admin to input stall information, such as stall name, specific canteen location, description, dietary restrictions and a stall image. A notable technical feature is the custom price input component that allows admin to input numerical value to define the relative price range. The UI translate the value into visual scale of dollar signs (e.g. “$$”) for consistent and easy visual display of price range as seen in Figure 37.

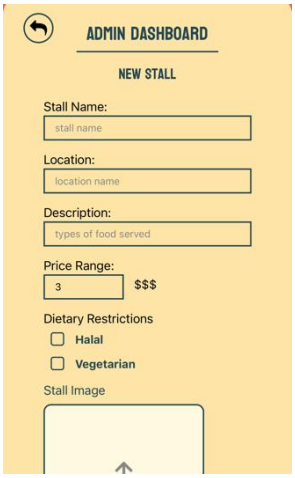


Figure 37. Admin Dashboard for Add New Stall

The “Stall List” interface acts as a directory for all the stalls in the application, where the admin can effectively manage the database, as shown in Figure 38. Each card consists of a “Edit” button and a “Delete” button. The “Edit” button is rendered green, signifying safe action. Interacting with this button routes admin to a pre-populated form with existing database values, allowing for rapid modifications. The “Delete” button is rendered red to signify destructive action. To mitigate human error and prevent accidental data loss, interacting with the button triggers a secondary verification prompt for admin before executing the deletion query.



Figure 38. Admin Dashboard for Stall List

4.8.2 Report Handling

To ensure the application remains a self-sustaining ecosystem, a reporting module was implemented. This approach relies on users to provide feedback to the admin to maintain data accuracy and safety within the application, reducing the need for constant manual administrative sweeps of the application.

The Report Page provides a platform for users to flag issues as shown in Figure 39. To maintain the friendly visual consistency, the header features a custom illustration of the NTUeats mascot on high alert while “making a report”, creating a softening the formal nature of reporting. The instructions below the illustration guides users to provide feedback, and the text input utilizes placeholder text to demonstrate a template for reporting.

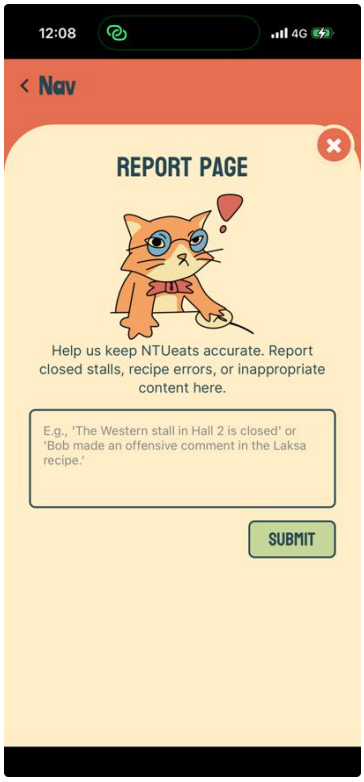


Figure 39. Report Page

The core innovation of this module lies in the integration of Gemini AI to convert the unstructured text of users into structured data for the administrators to see. The AI model has three main functions. It converts these unstructured texts into a concise summary, allowing administrators to understand the core issue with greater ease. The model also classifies the text into a specific Category, indicating

where the issue should be located at, such as Stall, Recipe, Comment. The model also assigns an Issue Type to the data, such as “Stall No Longer There” or “Inappropriate Content”.

The processed data is then displayed on the list in the report cards as shown in Figure 40, where it displays the AI-generated summary, original text and submission timestamp. The original text is added for reference as there may be possibility of vague summary or incorrectly translated ones.

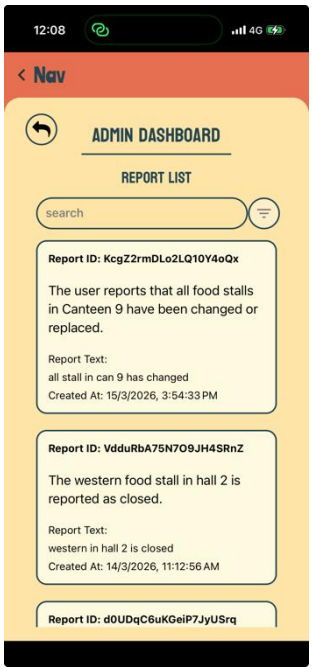


Figure 40. Admin Dashboard for Report List

To facilitate efficient database management, a search bar and filter function is implemented based on the AI’s categorization. The administrators can filter the reports by Issue Type and Category as shown in Figure 41.

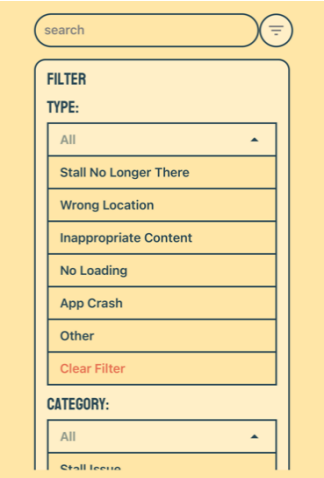


Figure 41. Filter options for Report List

This AI-driven pipeline provides the administrators with an organized view of the reports. It not only allows them to rapidly understand critical issues, but the filter also allows them to know where to tackle, hence streamlines the process in solving the problems. For example, they can immediately find and delete inappropriate content. This also facilitates targeted physical validation, where the administrators do not need to conduct routine checks at canteens for store closures, but instead deploy resources when they need to verify reports, reducing redundant checks.

4.9 Navigation Drawer

To facilitate mobility across the application, a Navigation Drawer is implemented and remains accessible from all pages. Users can deploy this Navigation Drawer through two interactions, and explicit tap target and an implicit gesture. A persistent “Nav” button anchored at the top-left of the screen triggers smooth panning to the drawer. To cater for one-handed usability, the drawer supports edge-swipe gestures, where users can pull the drawer by swiping the edge of the screen.

The layout of the drawer continues the application playful, character-driven design, where it features a custom illustration of the NTUeats mascot dressed as a sailor – a visual pun on “navigating” – accompanied by a Singlish prompt, “Looking for What?”. This maintains the brand’s localized and friendly tone.

Functionally, the drawer provides routing to the main modules and general pages, as shown in Figure 42. The navigation panel also allows admin to access the Admin Panel, where it employs conditional rendering to hide this route from users and to reveal it only to authorized administrators. Lastly, this drawer provides a Sign Out button at the bottom to facilitate account switching.



Figure 42. Navigation Drawer

4.10 Animation

To cultivate an interactive and engaging user experience, purposeful animations are implemented throughout the application. These animations serve to acknowledge user input and mask network latency.

The most repeated animation is the Touchable Scale button which dynamically scales the button on interaction. This provides an immediate acknowledgement to register user's input, giving a physical-like visual cue, as if the user is pressing an actual button.

To handle network latency, all images retrieved from the backend use a custom Image Loader component. This component uses a useState hook to monitor the image's "onLoadEnd" event to toggle between the image and loader. To keep with the application brand consistency, the component displays an 8-frame, low detailed animation of NTUeats mascot moving in a spiral pattern until the image loads, as shown in Figure 43. This transforms the mechanical wait-time into a playful moment, reducing the perceived latency.

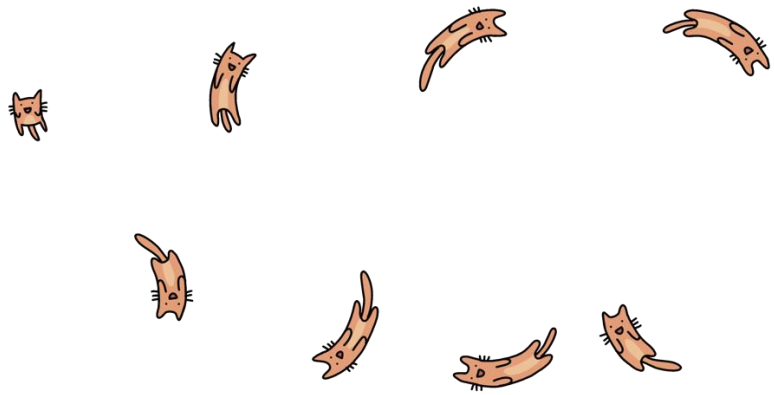


Figure 43. Mascot Loader Sprite Sheet

To facilitate smooth animation, React Native Reanimated and React Native Gesture Handler libraries are used for UI motions. These animations run on the UI thread, preventing frame drops even if the main JavaScript thread is busy.

One prominent implementation with these libraries includes the switching between general pages, where the outgoing pages fade to the right and incoming page slides into view from below. The Navigation Drawer uses the Reanimated Drawer Layout to create smooth and responsive panning when toggled. The code is shown in Figure 44.

```
<View className="bg-red h-screen realtive pt-4">
  <ReanimatedDrawerLayout
    ref={drawerRef}
    renderNavigationView={() => <NavPage closeDrawer={closeDrawer} />}
    drawerPosition={DrawerPosition.LEFT}
    drawerType={DrawerType.SLIDE}
    overlayColor="rgba(0, 0, 0, 0)"
    drawerWidth={300}
  >
    <GestureDetector gesture={tapGesture}>
      <NavButton />
    </GestureDetector>
    <Animated.View
      key={page}
      entering={SlideInDown.duration(800)}
      exiting={FadeOutRight.duration(800)}
    >
      {content}
    </Animated.View>
  </ReanimatedDrawerLayout>
</View>
```

Figure 44. Code for General Page Animation showing the animation applied

The file folders header in Home Page uses index-based state to dynamically adjust the width of navigation components. A Sequenced Transition is applied to the entire page to generate smooth transition between pages. The code is shown in the Figure 45.

```
{componentOrder.map(({ id, component: Component }, index) => (
  <Animated.View
    key={id}
    className="h-14"
    layout={SequencedTransition.duration(1000)
      .delay(200)
      .reverse()
      .reduceMotion(ReduceMotion.Never)}
  >
    <Component
      backgroundColor={index % 2 === 0 ? "darkcream" : "cream"}
      backgroundColorHex={index % 2 === 0 ? "#FFE6A7" : "#FFEFC7"}
      widthClass={
        index === 0
          ? "w-[14rem]"
          : index === 1
          ? "w-[12rem]"
          : "w-[10rem]"
      }
    />
  </Animated.View>
))}
```

Figure 45. Code for Home Animation showing the animation applied

Another prominent animation is applied to list components, such as the one for stall and recipe lists, where a Linear Transition is utilized to ensure smooth sorting of the cards when the filter updates the positions of cards.

4.11 React Hooks

To ensure a responsive user interface, the application leverages on React hooks to manage local states, side-effect handling and global context.

The useState hook handles the local state variables that ensure data in the user interface can be dynamically updated with data changes. This includes tracking of active indexes for sub-navigation, toggled states of buttons and holding data passed from the backend. This guarantees immediate and dynamic visual feedback upon user interaction.

The useEffect hook manages the component lifecycles and handles asynchronous operations. It allows for executing of database queries when the components mount and subsequently updates the local variable state with the data retrieved. It also provides the option to monitor specific variable state changes through dependency arrays and trigger targeted functions accordingly.

The useContext hook establishes a global state management architecture to allow certain state variables to be accessed throughout the application. This is to reduce prop drilling, where data is unnecessarily passed through multiple layers of components. The key global state includes dietary filters to synchronizes the restriction across all pages, and navigation state, to facilitate which pages are displayed. The navigation state consists of a “CurrentPage” element for specific page chosen and “SelectedID” element as a referential pointer for selected stall or recipe.

Chapter 5 Backend Development

5.1 Google Firebase

The application user Google Firebase for its backend services, which leveraged 3 different services to manage structured data, image storing and automate analysis of certain fields.

Cloud Firestore serves as a NoSQL database of the application, comprising of 15 collections as shown in Figure 46. Each collection consists of multiple documents, in turn holds multiple fields. The collection represents the groupings, document represents a single record, and field represents a data point.

To create an intuitive workflow for developers, the collections adopt a specific naming convention. Collections tagged with the "_likes" and "_saved" functions as a join table, mapping user IDs to their respective liked stall, images or comments. Collection tagged with the "_dietary" tag maps the specific stall, recipe or user with their respective restrictions.

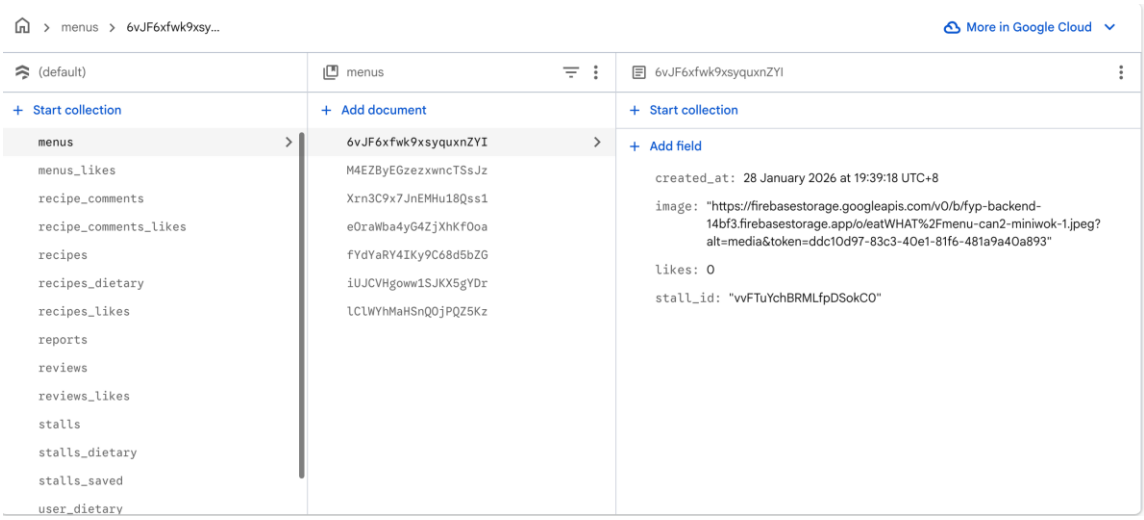


Figure 46. Cloud Firestore database

Firebase Storage manages the images in the application and split them into 2 groups – EatWHAT and CookWHAT – to organize the data. A strict naming convention was adopted for intuitive reference. For data in EatWHAT, they are labelled "type - location - name - number", where type refers to whether the images are menus, images from review or stall images, location and name refers to which canteen the stall is located at and the name of the stall respectively, and number refers to the index of data

added. For data in CookWHAT, they are labelled “type - name - number”, where type refers to whether the images are recipes or comment, name refers to the name of food in the recipe and number refers to a timestamp for recipe – to keep it unique, and index for comment. The images are then linked to the database via a URL link with access token. The example for EatWHAT is shown in Figure 47.

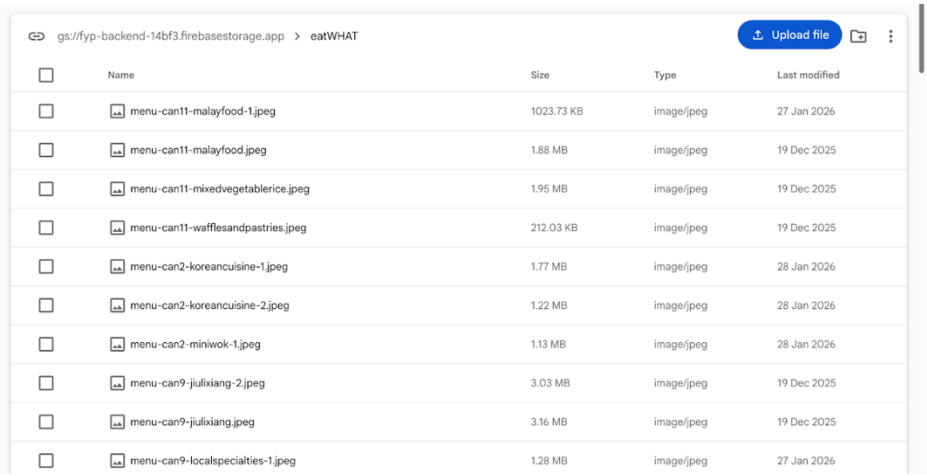


Figure 47. Image storage for EatWHAT

Multimodal Tasks with the Gemini API enables the Google AI model to process the user reports, integrating with Gemini 3 Flash model. The prompt sent to the AI mode is as follows:

“ You are a strict JSON classifier; analyze {{text}} and return exactly one valid JSON object (no markdown/explanations) with keys in order category, type, summary, where category ∈ ["stalls", "stall review", "recipe", "recipe comment", "technical bug", "profile", "invalid", "others"], type ∈ ["stall no longer there", "wrong location", "contains inappropriate content", "no loading", "app crash", "spam"], summary is exactly one concise factual sentence, and if input is empty/unclear/gibberish use {"category": "invalid", "type": "spam", "summary": "The report is unclear or invalid."}. “

When a new report is created in Firestore, the extension triggers the AI model to analyze the raw input. The AI interprets the unstructured report text and generates an “analysis” field, containing interpreted Category, Issue Type and Summary as shown in Figure 48, allowing for filtering as mentioned in the admin page.

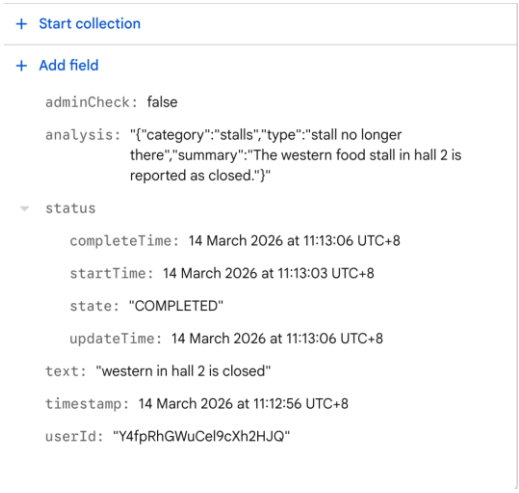


Figure 48. Example of report document, where text refers to the report text input by user and the analysis refers to the interpreted output generated from the AI model.

5.2 Backend Architecture

A Service Layer was developed to manage interactions between the frontend and Cloud Firestore. This architecture follows the CRUD (Create, Read, Update, Delete) pattern for interacting with all collections in the database. This ensures data can be created, updated or deleted by the user, and data can be received and displayed on the frontend interface. The list of utilities is shown in Figure 49, each representing a targeted collection.

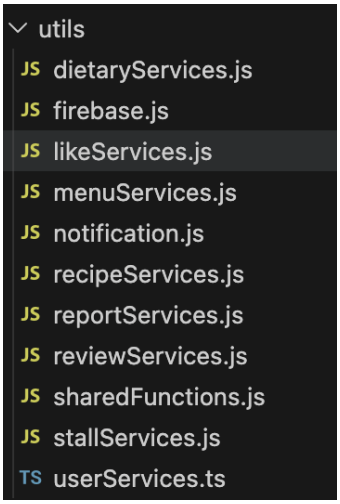


Figure 49. Backend services utilities

The services handle limiting, sorting and filtering functionalities through parameter-driven queries, where the functions accept multiple input constraints and dynamically adjusts the query and data collected from the backend. For example, the “Stall Getter” service handles the sorting algorithm through multiple if-else logic arrangement algorithms. It also filters the data from search terms and dietary restrictions, displaying only relevant data, and limits the number of documents retrieved to optimize data collection. These algorithms are shown in Figure 50.

```
// Fetch stalls with restrictions applied if restrictions exist, otherwise get all stalls.
const stallsData = Array.isArray(resolvedOtherData)
  ? await getStallsWithRestrictions(restrictions, resolvedOtherData) // use pre-filtered data if provided (e.g., from search results)
  : await getStallsWithRestrictions(restrictions);

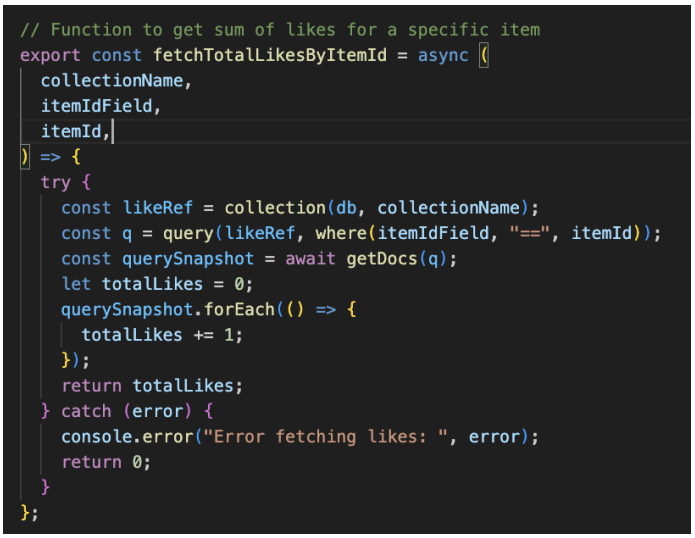
// Then arrange the filtered stalls based on the selected arrangement
let arrangedStalls = [...stallsData];
if (arrangement === "most_saved") {
  arrangedStalls.sort((a, b) => b.saves - a.saves);
} else if (arrangement === "alphabetical") {
  arrangedStalls.sort((a, b) => a.name.localeCompare(b.name));
} else if (arrangement === "price_low_to_high") {
  const priceOrder = { $: 1, $$: 2, $$$: 3, $$$$: 4 };
  arrangedStalls.sort(
    (a, b) => priceOrder[a.price_symbol] - priceOrder[b.price_symbol],
  );
} else if (arrangement === "price_high_to_low") {
  const priceOrder = { $: 1, $$: 2, $$$: 3, $$$$: 4 };
  arrangedStalls.sort(
    (a, b) => priceOrder[b.price_symbol] - priceOrder[a.price_symbol],
  );
} else {
  if (typeof resolvedRandomSeed === "string" && resolvedRandomSeed) {
    arrangedStalls.sort((a, b) => {
      const hashA = getDeterministicHash(`${resolvedRandomSeed}:${a.id}`);
      const hashB = getDeterministicHash(`${resolvedRandomSeed}:${b.id}`);
      return hashA - hashB;
    });
  } else {
    arrangedStalls.sort(() => Math.random() - 0.5);
  }
}

// Get length before slicing for limit
const length = arrangedStalls.length;
// Apply limit if specified
if (typeof limitNum === "number" && limitNum > 0) {
  arrangedStalls = arrangedStalls.slice(0, limitNum);
}

return { data: arrangedStalls, length };
```

Figure 50. Code snippet of StallServices.js that shows the algorithms within the getter function

To improve maintainability, certain services are created to target multiple tables. For example, “DietaryServices” contains functions to validate, toggle and filter content based on dietary restrictions. “LikeServices” contains function to handle counting, updating and checking likes for the various entities. These services include a generic template for parameter, requesting for collection name of the desired table and item id field that the function to search for as shown in Figure 51.



```
// Function to get sum of likes for a specific item
export const fetchTotalLikesByItemId = async (
  collectionName,
  itemIdField,
  itemId,
) => {
  try {
    const likeRef = collection(db, collectionName);
    const q = query(likeRef, where(itemIdField, "==", itemId));
    const querySnapshot = await getDocs(q);
    let totalLikes = 0;
    querySnapshot.forEach(() => {
      totalLikes += 1;
    });
    return totalLikes;
  } catch (error) {
    console.error("Error fetching likes: ", error);
    return 0;
  }
};
```

Figure 51. Code snippet for LikeService.js for fetching total likes for an entity

The services also include other functionalities, such as handling transformation of certain data like timestamps form the database to human-readable format for the frontend and creating hashes for randomizer.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

NTUeats was conceptualized for the purpose of solving the question of what to eat and promoting food discovery within campus. The primary objective was to develop a self-sustaining application that requires minimal administrative maintenance. Through the various iterations and development, it has managed to fulfil core functionalities of stall review and recipe sharing.

Additional features were developed such as users' ability to manage their own bookmarks, uploads and dietary preferences. Functionalities, such as search, filter and randomizer, were also created to help users. Furthermore, implementation of a dedicated administrative dashboard has allowed for management of stall data, and AI-powered reporting ensures administrators can efficiently target important issues.

Beyond its functional features, the focus of the application was to create a fun and unique experience through playful design aesthetics, thoughtful animations and familiar, localized Singlish phrases. Also, the character-driven focus was established with numerous playful illustrations of the NTUeats mascot. This intentional blend of functional development and design elements ensures that the platform is not only useful but also engaging and creates a sense of community and joy within the NTU community.

The overall experience of designing and developing the application was rewarding, where I learnt not only to develop, but to debug issues and refine complex animations. The invaluable skills were discovered through designing on Figma to developing in React Native. The most significant challenge involved migrating the database from an initial Neon setup to Firebase ecosystem to optimize image handling and to leverage on the serverless and secure backend, and to allow for frontend-focused development.

6.2 Recommendation in Future Work

Due to time constraints, the project has not reached User Testing phase, which involves identifying pain points and improvement for the application. Also, the iteration of the project is proof of concept and does not have a complete list of all the stalls and canteens in NTU. A one-time sweep through of all canteens at NTU and adding all stalls can help further complete the application.

Moreover, the development of the application can be further extended to implement the following features:

1. Current iteration of the application has not implemented functionality for changing passwords and email, and the section in the profile serves a temporary placeholder for it, and future development can work on it.
2. Future iterations should allow the administrators to directly delete specific comments, recipes and reviews that violate community guidelines, and issue warnings or ban specific users.
3. Development of a page that shows specific user's recipes may also be good, especially for users that favor a certain publisher's style of cooking.
4. Like other social media algorithm, an AI model that analyzes user's interaction can suggest "randomized" stalls and recipes that are tailored to individual taste rather than a basic randomizer.
5. An AI interpreter of the menu images can also allow for easy searching of dishes instead of searching by stall name alone.
6. To ensure students are the only ones that post review and share recipes, serving the purpose that "students know other students' needs best", future implementations can incorporate NTU email validation to the application.

Reflection on Learning Outcome Attainment

Engineering knowledge: I had learned essential software development knowledge. I had learnt to master React's Hook functionality, Firebase API structure, animations libraries for React Native, and many other tools. I also applied engineering workflow through systematic debugging, where I tested the written code with bugs through console logging and isolating certain variables to identify the reason for problems, followed by iterative testing to fix the issue.

Design/development of Solutions: Developing the application from scratch required planning from many different aspects, including designing the aesthetic, planning for the layout of individual pages and creating low-fidelity prototypes. The development of translating the Figma layout to code shows the implementation of development of solution.

Modern Tool Usage: A modern tech stack was used for development, comprising of React Native with Expo for frontend development and Google Firebase for backend development. Moreover, the use of Gemini AI via the Multimodal Tasks extension uses the AI model as a functional tool for transforming data. This is the beginning of understanding modern tools, as technology rapidly grows due to the advent of Artificial Intelligence, but serves as a good foundation for understanding current trends.

Lifelong Learning: The most important lesson in this development process is self-directed learning in mastering new technology. I had gone to learn the modern tech stack without guidance, and trialed and tested new libraries for the application. It really allowed me to practice referencing and searching online resources, learning from YouTube tutorials, and getting information from various sources to develop an application from scratch. This unguided practice would be the foundation for my future learnings outside of school.

However, I also learnt the difference between the various databases too late and made the decision to completely migrate from Neon to Firebase, allowing me to further understand the pain of such decisions. In development, it is important to do research and plan out the necessary functionalities that are required for the project before implementation, but I had rushed into building a backend early, teaching me the importance of thorough planning in the future.

References

[1] D. O. Mensah and O. Oyebode, "'We think about the quantity more': factors influencing emerging adults' food outlet choice in a university food environment, a qualitative enquiry," *Nutrition Journal*, vol. 21, no. 1, Art. no. 49, Mar. 2022, doi: 10.1186/s12937-022-00801-0.

[2] T. Schubert, "People choose options that leave future options open," *Nature Reviews Psychology*, vol. 2, no. 3, pp. 135–135, Mar. 2023, doi: 10.1038/s44159-023-00164-1.

[3] J. Leu, S. A. Rebello, G. M. Sargent, M. Kelly, and C. Banwell, "The Singaporean foodscape - Convenience, choice, entertainment, and sociality for young workers," *Appetite*, vol. 189, Art. no. 106992, Feb. 2023, doi: 10.1016/j.appet.2023.106992.

[4] O. Wong, "Residents share: Eating, cooking, and what it means to them," *NUS Residential Life*. Accessed: Sept. 19, 2025. [Online]. Available: https://blog.nus.edu.sg/reslife/2023/04/16/residents-share-eating-cooking-and-what-it-means-to-them/.

[5] "General Directory," *Nanyang Technological University*. Accessed: Feb. 21, 2026. [Online]. Available: https://www.ntu.edu.sg/life-at-ntu/leisure-and-dining/general-directory?locationTypes=all&locationCategories=all&page=1.

[6] "NTU Canteens," *Google Map*. Accessed: Feb 21, 2026. [Online]. Available: https://www.google.com/search?q=ntu+canteens

[7] Wecome Pte Ltd, Wecome Cates, Ver 1.3.8, Singapore: Wecome Pte Ltd, 2026.

[8] B. Vaghela , "Benefits of React Native for mobile app development," *IEEE Computer Society*. Accessed: Mar. 12, 2026. [Online] Available: https://www.computer.org/publications/tech-news/trends/benefits-of-react-native.

[9] V. Vaishnav, "Why Use Firebase," *GeeksforGeeks*. Accessed: Mar. 12, 2026. [Online] Available: https://www.geeksforgeeks.org/firebase/why-use-firebase/.

[10] "Using Singlish in Mobile Apps UX? Seriously Ah?," *Originally US*. Accessed: Mar. 12, 2026. [Online] Available: https://originallyus.sg/using-singlish-in-mobile-apps-ux-seriously-ah/.

[11] E. D. J. Salamena, “The right color for culinary branding based on psychology,” *BPSY Telkom University*. Accessed: Mar. 12, 2026. [Online] Available: https://bpsy.telkomuniversity.ac.id/en/the-right-color-for-culinary-branding-based-on-psychology/

[12] Lauren, “The Color Cream | Meaning and Psychology,” *BrandCrowd*. Accessed: Mar. 12, 2026. [Online] Available: https://www.brandcrowd.com/blog/the-color-cream-meaning-and-psychology

[13] Reddy, B. V. V, Y. Hemanth, K. S. Ganesha, K. Jayashree, N. Amruta, and R. Saravanan, “The role of mascots in branding and consumer engagement: Insights from Indian advertising,” *Multidisciplinary Reviews*, vol. 8, no. 5, Art. no. 2025151, 2025, doi: 10.31893/multirev.2025151.

[14] D. H. Duncan, N. Forschack, D. V. Moorselaar, M. M. Muller, and J. Theeuwes, “Learning Modulates Early Encephalographic Responses to Distracting Stimuli: A Combined SSVEP and ERP Study,” *Journal of Neuroscience*, vol. 45, no. 21, May 2025, doi: 10.1523/JNEUROSCI.1973-24.2025.